

lec-29

Rö

Recursion - pattern questions + Bubble sort + Selection sort

Q₁ Triangle

Print this
triangle

```
4 * * * *
3 * * *
2 * *
1 *
```

* for this we need to loop's one is for row and another is for column

fun(r, c) | initially = (4, 0)

(4, 0) → *

↓

(4, 1) → * *

↓

(4, 2) → * * *

↓

(4, 3) → * * * *

↓

new line

↓

(3, 0) → * * * *

*

Steps:-

- i) initially $(r, c) = (4, 0)$
- ii) check if $r < c$ if not the print *, Repeat the steps until you condition violated.
- iii) whenever your condition is violated then decrement the row value in a new line and $c = 0$
- iv) Repeat the steps till $r == 0$.

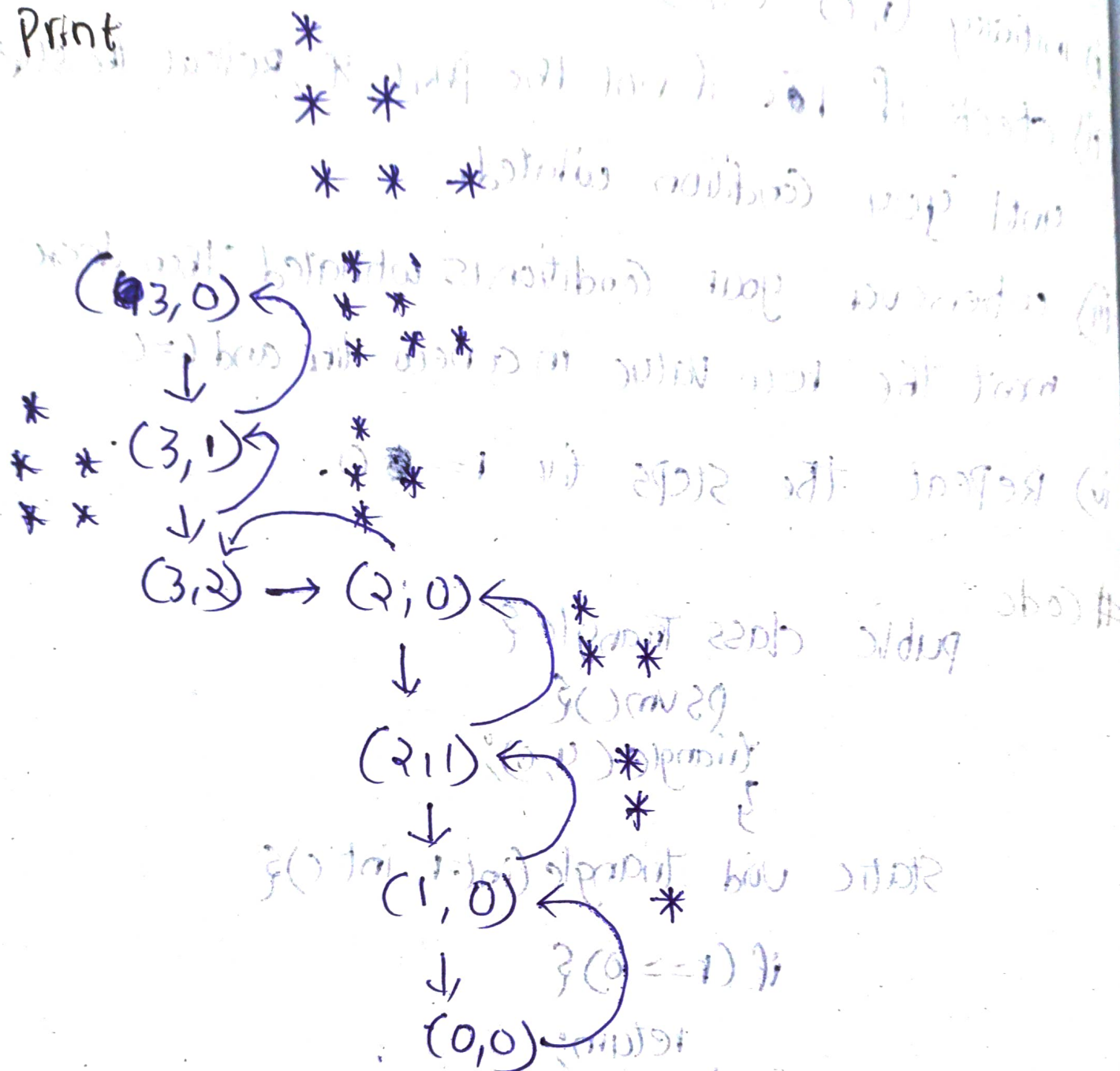
#Code

```
public class Triangle {  
    psvm() {  
        triangle(4, 0);  
    }  
    static void triangle(int r, int c) {  
        if (r == 0) {  
            return;  
        }  
        if (c < r) {  
            sout("*");  
            triangle(r, c+1);  
        } else {  
            sout("\n");  
            triangle(r-1, 0);  
        }  
    }  
}
```

Q. Triangle 2

B8

Print



Steps:

- i) initially $(r, c) = (3, 0)$ i.e.
- ii) Check if $r > c$ if not then increase the value of c till then put the value in the stack.
- iii) When ever the ~~stack~~ element removing from the stack print at that time.

#code

```
public class Triangle {  
    psum() {
```

```
        triangle(3, 0);
```

```
    static void triangle(int r, int c) {
```

```
        if (r == 0) {
```

```
            return;
```

```
        if (r > c) {
```

```
            triangle(r, c+1);
```

```
            cout << " * ";
```

```
        } else {
```

```
            triangle(r-1, 0);
```

```
            cout << endl;
```

Q3. Bubble Sort

↳ It is the simplest algorithm that works by repeatedly swapping the adjacent element if they are in wrong order

→ For every pass the biggest element will be at last

index

(4, 3, 2, 1) (3, 0)

(3, 4, 2, 1) (3, 1)

(3, 2, 4, 1) (3, 2)

(3, 2, 1, 4) (2, 0)

steps

(4, 3, 2, 1)
↔

(3, 0)
↓

check in my third row first element
is smaller or bigger than the next element
if bigger then swap that

(3, 4, 2, 1)

(3, 1)

→ Increase the column value till $r > c$ whenever
it is validated then decrement value of row and
column as 0

** This is similar to the q. pattern this lecture only

#code public class bubbleSort {

static void bubble (int[] arr, int r, int c) {

if (r == 0) {

return;

}

if (c < 1) {

if (arr[c] > arr[c+1]) {

int temp = arr[c];

arr[c] = arr[c+1];

arr[c+1] = temp;

}

bubble (arr, r, c+1);



```
} else {  
    bubble(arr, r-1, 0);  
}
```

```
} }
```

```
psvm() {
```

```
int[] arr = {4, 3, 2, 1};
```

```
bubble(arr, arr.length-1, 0);
```

```
out(Arrays.toString(arr));
```

```
}
```

→ Here the array in every recursion call pointing to the same object so whenever you modify the arr the original array will also change

Q4. Selection Sort

↳ get the greatest element of the array & put it on its correct index.

Ex: 4, 3, 2, ⑧, 1

↓ (8 is max)
⑧, 3, 2, 1, 4

↓
1, ⑧, 2, 3, 4

↓
1, 2, ⑧, 3, 4

↓
1, 2, 3, 4, ⑧

* It is similar to the q, but here when we are adding a new line that means we have ~~seen~~ traversed the entire array and we have find the largest element at that time swap this with ~~last~~ index

how to check the max element in every row?

flow: arr = (4, 3, 8, 1) (4, 0, 0) ^{↑ r ↑ c ↑ max}

Hey is my first element which is at '0' index is greater than the max no then swap it

arr = (4, 3, 8, 1) (4, 1, 4)

Hey is 3 > 4, no, then increment 'c'

arr = (4, 3, 8, 1) (4, 2, 4)

Hey is 8 > 4 yes swap max as 8 and increment 'c'

arr = (4, 3, 8, 1) (4, 3, 8)

Hey is 1 > 8, no, no need to do any thing just increment 'c'

(4, 4, 8)

Here the r and c are equal condition satisfied

So decrease the r and c
r-1

#Code

```
public class selection {
```

```
    public static void main ( ) {
```

```
        int [] arr = { 4, 2, 3, 1 };
```

maintain max as index
0

```
        select (arr, arr.length-1, 0, 0);
```

```
        System.out.println (Arrays.toString (arr));
```

```
    }
```

```
static void select (int [] arr, int r, int c, int max) {
```

```
    if (r == 0) {
```

```
        return;
```

```
    }
    if (c < r) {
```

```
        if (arr[c] > arr[max]) {
```

```
            select (arr, r, c+1, c);
```

```
        } else {
```

```
            select (arr, r, c+1, max);
```

```
        }
    }
    if (c < r) {
```

```
        int temp = arr[max];
```

```
        arr[max] = arr[r-1];
```

```
        arr[r-1] = temp;
```

```
        select (arr, r-1, 0, 0);
```

```
    }
```

swapping
with last
index